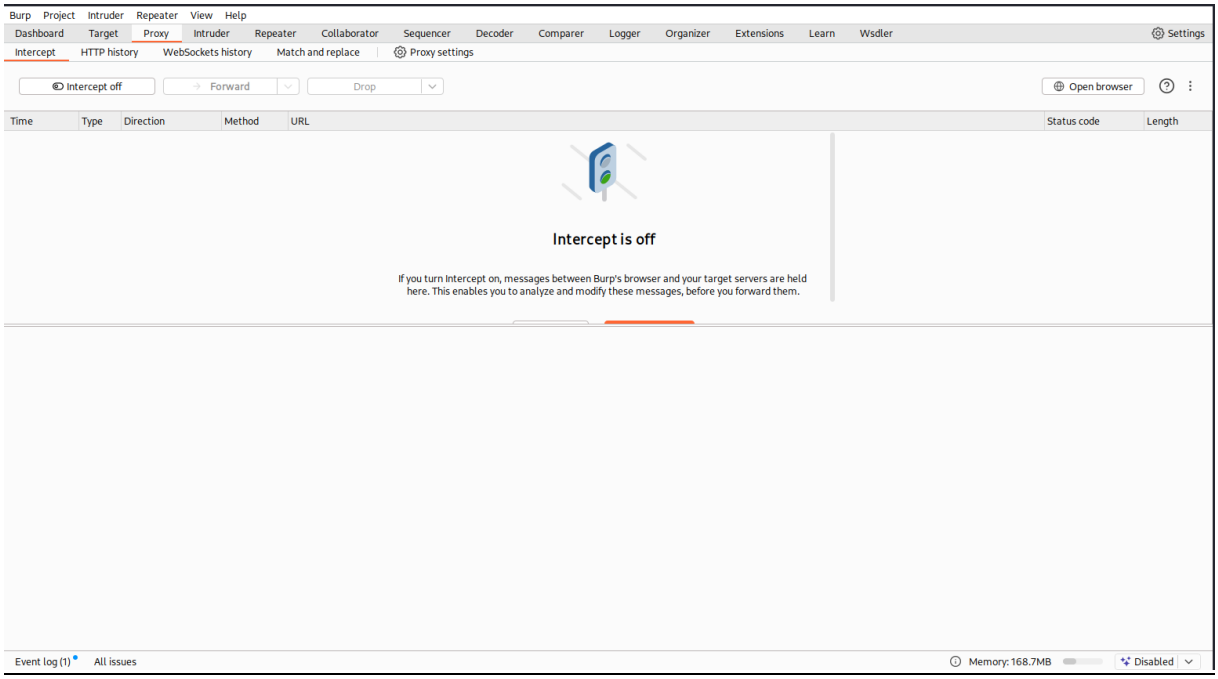
Activité 4 : Brèche sur des informations confidentielles



Burp Suite est un outil utilisé pour tester la sécurité des applications web. Il permet d'analyser les échanges entre le navigateur et le serveur afin de détecter des failles de sécurité.

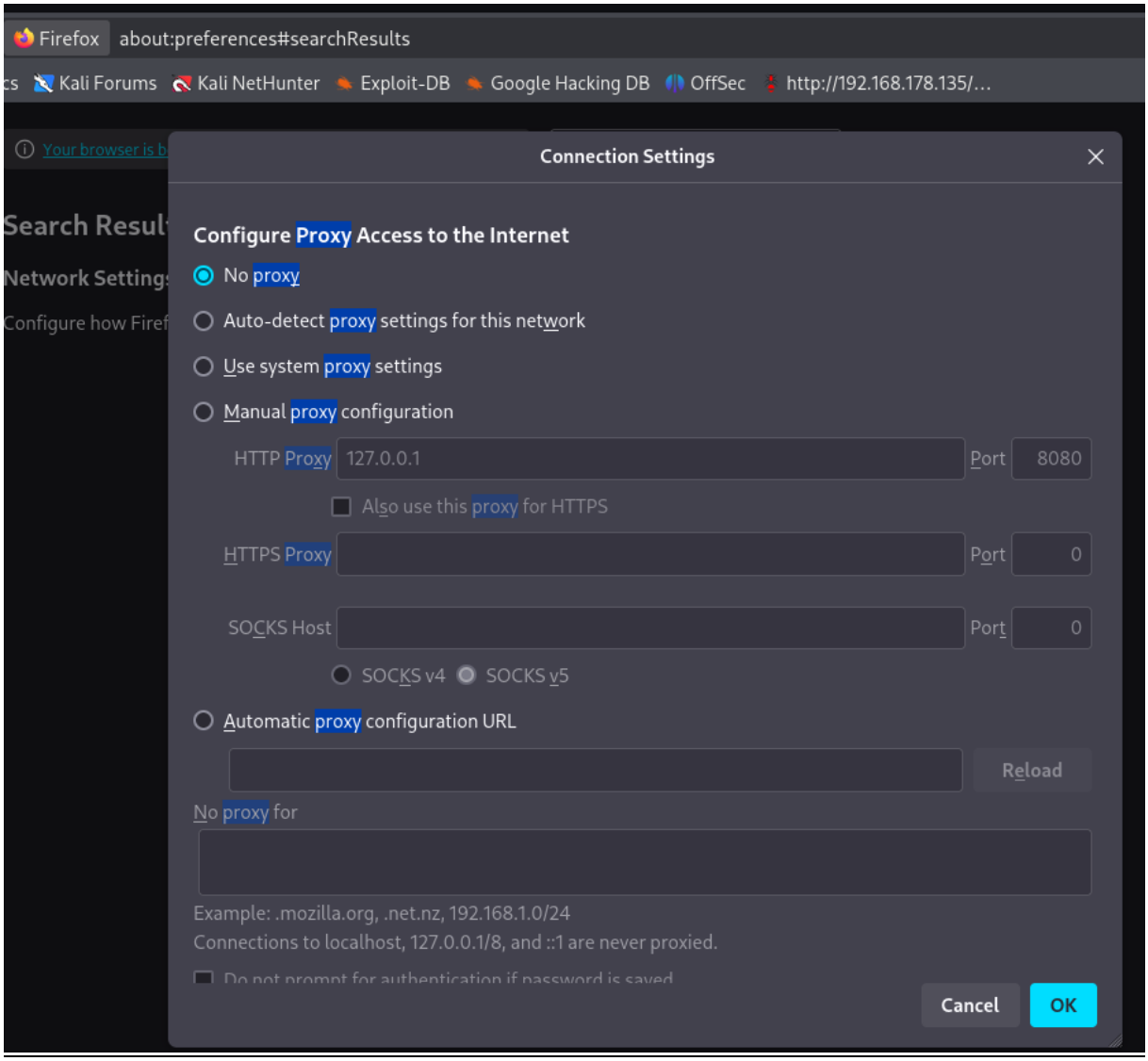
Grâce à l'activation du proxy, Burp Suite intercepte les requêtes HTTP et HTTPS envoyées par le navigateur. Cela permet de voir, modifier ou rejouer les requêtes avant qu'elles n'arrivent au serveur.

L'onglet Repeater permet de renvoyer manuellement une requête autant de fois que nécessaire, en modifiant certains paramètres, afin d'observer le comportement de l'application.

L'outil Intruder sert à automatiser des attaques en envoyant un grand nombre de requêtes avec différentes valeurs (par exemple pour tester des injections ou des mots de passe).

L'onglet Comparer permet de comparer plusieurs réponses du serveur afin d'identifier des différences, ce qui est utile pour analyser le fonctionnement de la sécurité.

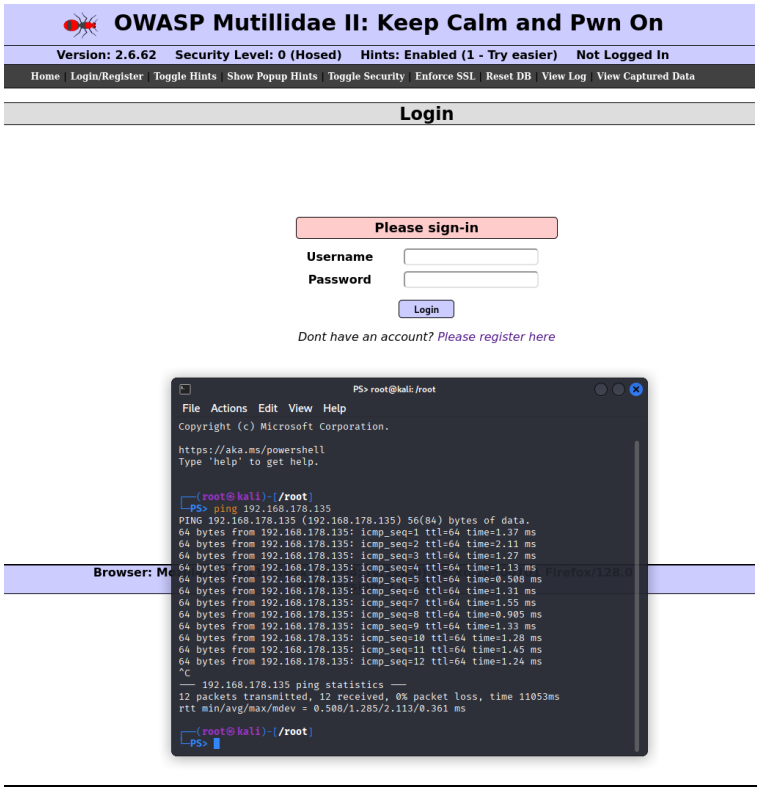
Burp Suite est donc un outil complet pour tester, comprendre et analyser la sécurité d'une application web.



Dans la machine virtuelle, il a également été nécessaire de configurer le navigateur. Pour cela, il a fallu accéder aux paramètres en allant sur about:preferences. Ensuite, l'option No proxy a été désactivée afin de permettre au navigateur d'utiliser le proxy de Burp Suite. Cette configuration est indispensable pour que Burp Suite puisse intercepter et analyser les requêtes envoyées par le navigateur.

Travail à faire 1 :

Q1 :



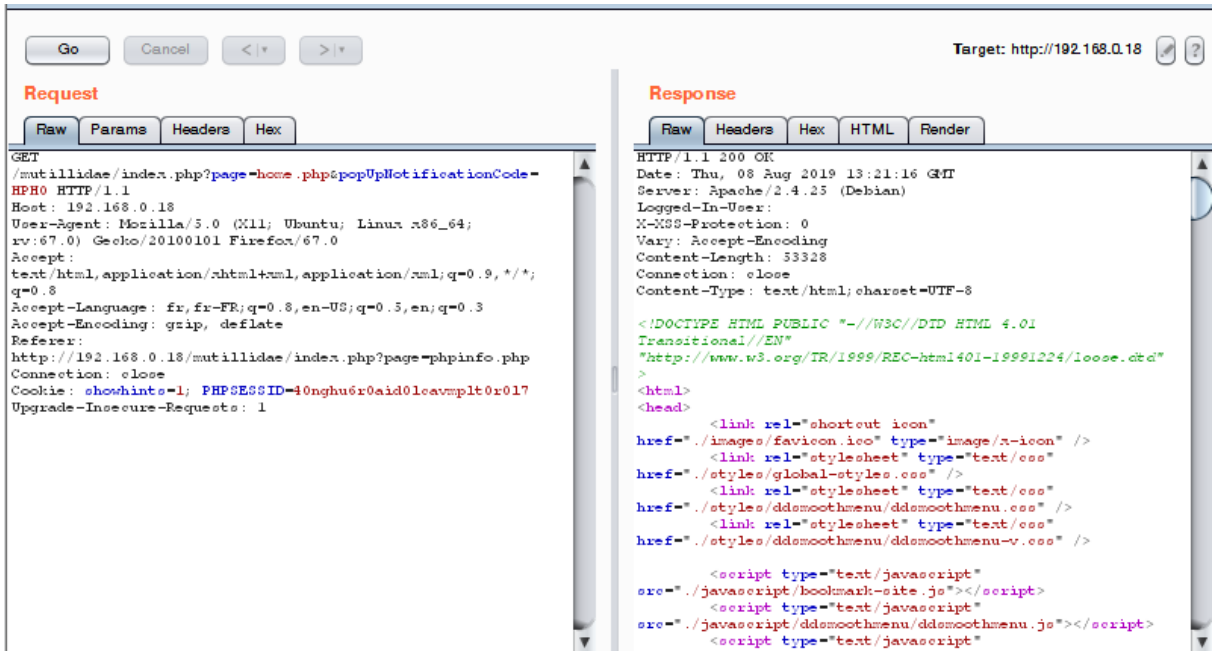
Les deux machines virtuelles ont été démarrées correctement : le serveur Mutillidae et la machine cliente équipée de BurpSuite. La communication entre les deux machines a été vérifiée à l'aide de la commande ping, confirmant que le réseau fonctionne correctement. Le niveau de sécurité de Mutillidae a ensuite été positionné à 0 afin de permettre la réalisation des tests de sécurité.

Q2 :

Vérifier que le proxy BurpSuite est bien en mode « Intercept is on ». Puis aller sur la page d'accueil de Mutillidae. La requête de connexion est alors interceptée par le proxy.

Il s'agit d'une page valide qui existe réellement sur le serveur. Le but est de comparer la réponse envoyée par le serveur entre une page existante et une page qui n'existe pas.

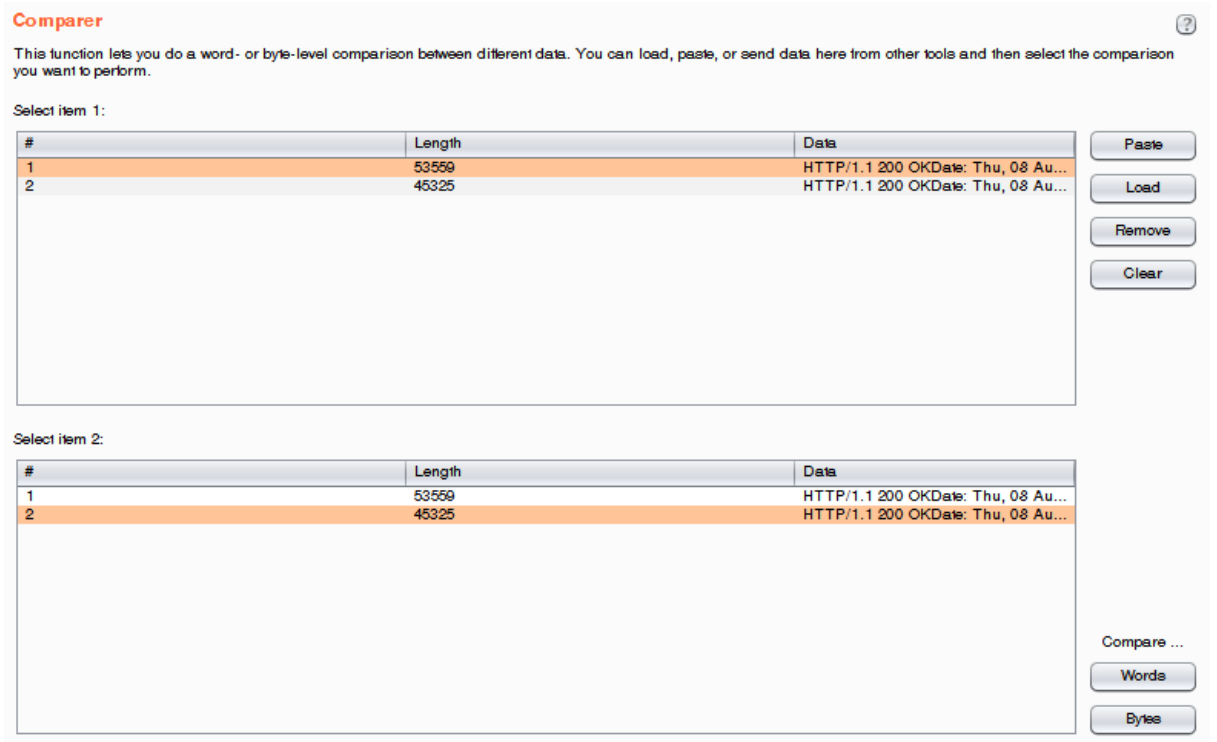
Faire un clic droit dans la fenêtre d'interception puis sélectionner « Send to Repeater ». Aller ensuite dans l'onglet Repeater et cliquer sur le bouton « Go » afin d'afficher la réponse de la page existante home.php.



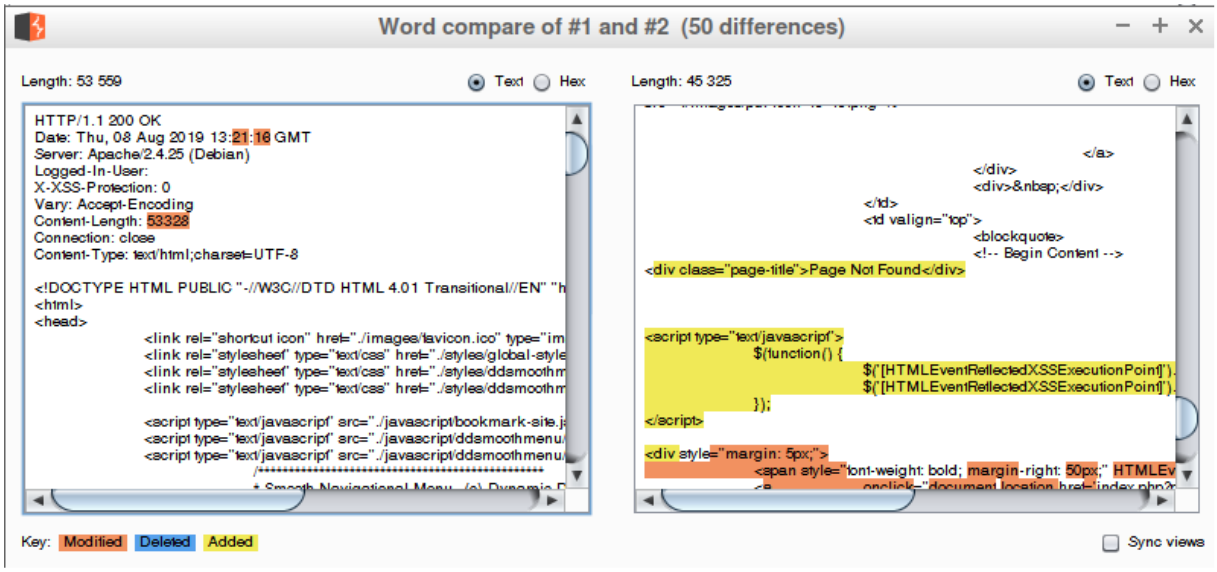
Faire ensuite un clic droit au centre de la réponse affichée (partie droite) et choisir « Send to Comparer ».

Puis refaire toutes les étapes précédentes avec une page qui n'existe pas sur le serveur, par exemple home777.php. Envoyer également la réponse du serveur

vers le comparateur en sélectionnant « Send to Comparer ». Vous devriez alors obtenir le même résultat dans l'onglet Comparer de BurpSuite.



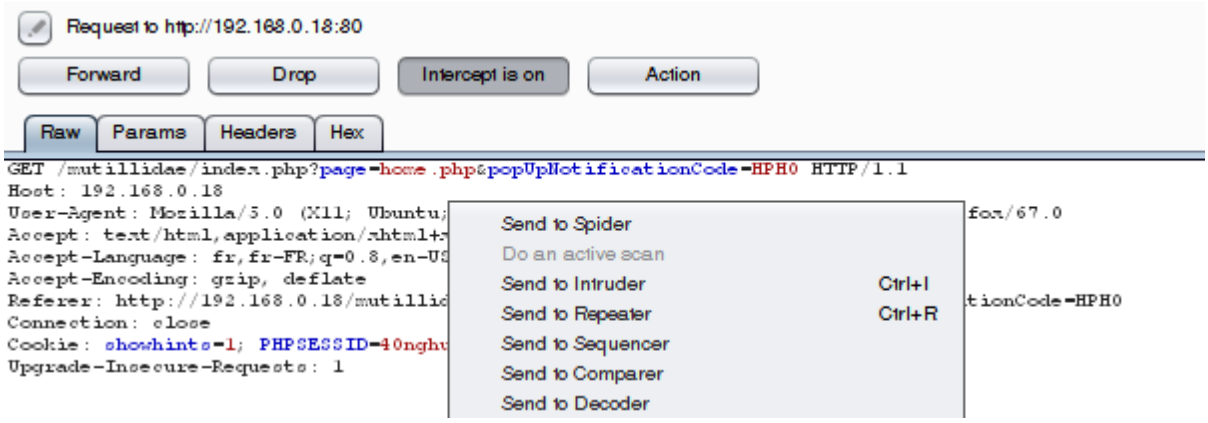
Cliquer ensuite sur le bouton « Words » en bas de la fenêtre du comparateur, puis repérer l'affichage de la phrase qui indique l'erreur 404 (Page Not Found).



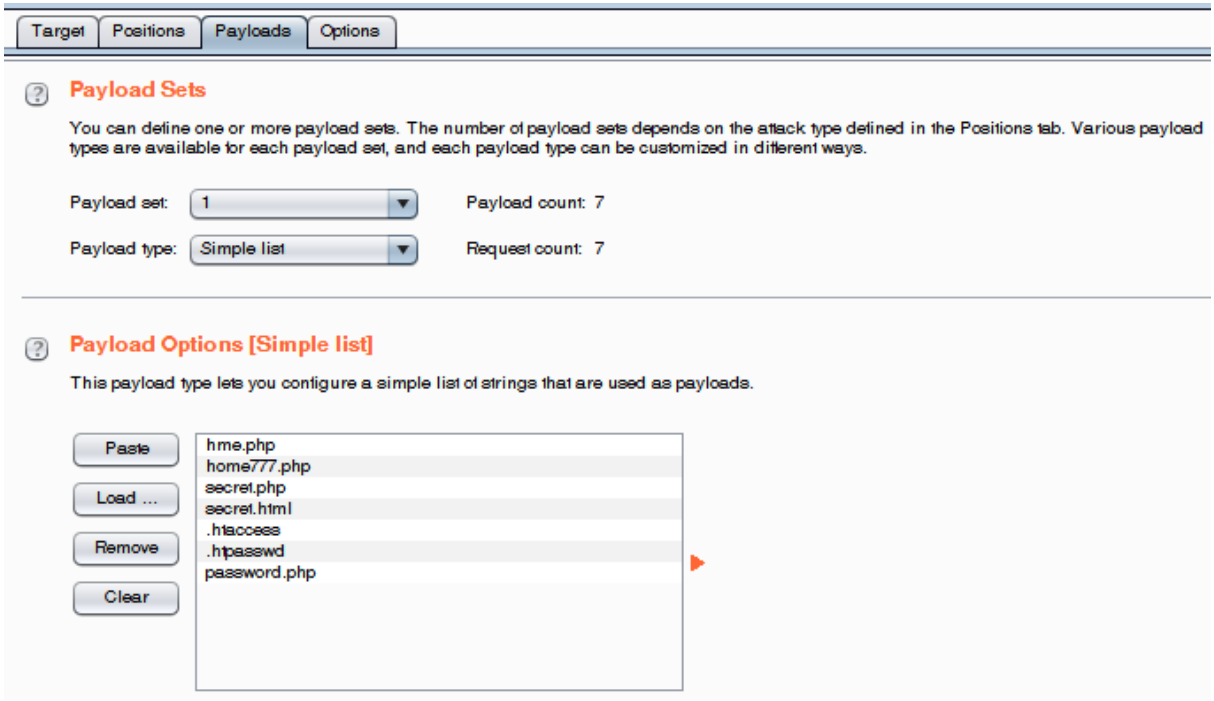
C’est sur cette chaîne de caractères que nous allons nous baser pour analyser les pages du site et vérifier, à l’aide d’un dictionnaire de noms de pages, si elles existent ou non.

Ce dictionnaire contient des noms de pages courants liés aux configurations (*.htaccess, *.htpasswd*, etc.).

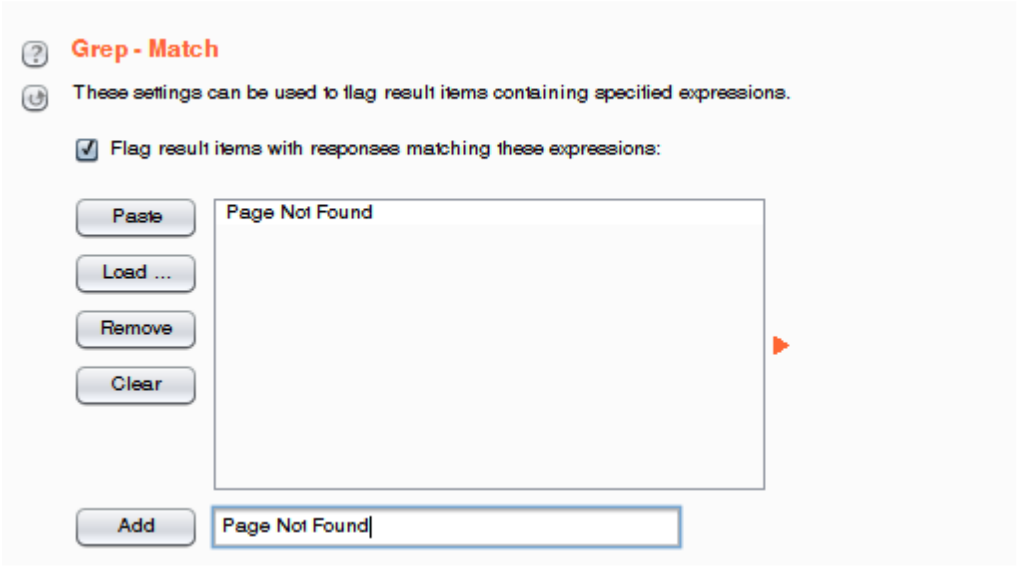
Il s’agit donc d’une attaque par dictionnaire. Cette méthode est appelée le fuzzing de pages Web.



Aller dans l’onglet Intruder. Dans l’onglet Positions, cliquer sur le bouton Clear, puis sélectionner le nom de la page home.php en double-cliquant dessus. Ensuite, cliquer sur le bouton Add afin d’utiliser le nom de la page comme paramètre pour le fuzzing. Le type d’attaque reste configuré par défaut sur Sniper.



Ensuite, se rendre dans l'onglet Options, dans la section Grep Match. Il faut commencer par vider le contenu existant en cliquant sur le bouton Clear. Puis, ajouter la chaîne de caractères Page Not Found dans le champ prévu à cet effet. Ce filtre permettra d'identifier plus facilement les réponses correspondant à cette erreur.



Il ne reste plus qu'à lancer l'attaque en cliquant sur le bouton Start Attack. Les pages qui ne sont pas cochées dans les résultats correspondent aux pages existantes sur le serveur.

Results | Target | Positions | Payloads | Options

Filter: Showing all items

Request	Payload	Status	Error	Timeout	Length	Page N...
0		200	☐	☐	53559	☐
1	hme.php	200	☐	☐	45335	☑
2	home777.php	200	☐	☐	45359	☑
3	secret.php	200	☐	☐	140666	☐
4	secret.html	200	☐	☐	45359	☑
5	.htaccess	200	☐	☐	140646	☐
6	.htpasswd	200	☐	☐	140649	☐
7	password.php	200	☐	☐	45365	☑

Ensuite, double-cliquer sur la ligne correspondant à la page .htpasswd. Puis, se rendre dans l'onglet Response et dans le sous-onglet Render afin d'afficher le contenu de la réponse.

Result 6 | Intruder attack 1

Payload: .htpasswd
Status: 200
Length: 140649
Timer: 38

Previous
Next
Action

Request | Response

Raw | Headers | Hex | HTML | Render

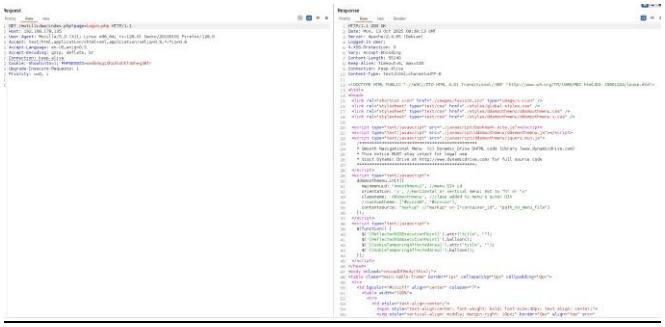
Directive	
allow_url_fopen	On
allow_url_include	Off
arg_separator.input	&
arg_separator.output	&
auto_append_file	no value
auto_globals_jit	On
auto_prepend_file	no value

Il est alors possible d'afficher la page phpinfo, qui fournit des informations importantes sur la configuration du serveur. Cette page est accessible lorsque le niveau de sécurité est réglé sur 0, ce qui correspond à une fuite d'informations

sensibles. En revanche, avec un niveau de sécurité plus élevé, comme le niveau 5, l'accès à cette page n'est plus autorisé.

Travail à faire 2 :

Q1 :



Le niveau de sécurité 1 réduit quelques risques mais ne supprime pas forcément la fuite d'information. Il ajoute des protections basiques qui peuvent bloquer des attaques simples. La solution sûre est de désactiver ou supprimer les pages de debug et d'ajouter un contrôle d'accès (authentification).

Q2 :

En observant le code source de la page phpinfo.php, on voit qu'elle exécute directement la fonction phpinfo() sans vérifier si l'utilisateur est connecté ou autorisé. Quand le niveau de sécurité est à 0, le serveur affiche la page librement, ce qui provoque une fuite d'informations sensibles sur la configuration du serveur.

En revanche, au niveau 1 le code ajoute généralement une vérification ou un contrôle d'accès avant d'exécuter phpinfo(). Si cette vérification détecte qu'aucun utilisateur n'est authentifié, l'accès est refusé ou redirigé vers la page de connexion.

Cela explique pourquoi la page devient partiellement ou totalement inaccessible à ce niveau.
Ainsi, le serveur réagit mieux face à l'attaque, car il empêche la consultation directe du fichier.
Cependant, la protection reste simple et pourrait être contournée si le code ne filtre pas correctement tous les cas.

Q3 :

Le niveau de sécurité 5 offre des protections beaucoup plus strictes et a de fortes chances d'empêcher l'accès direct à phpinfo().

À ce niveau, Mutillidae applique des contrôles d'accès renforcés, un filtrage plus strict et la désactivation ou l'obfuscation des pages de debug.

Si phpinfo.php est correctement protégée ou retirée, la fuite d'informations ne sera plus visible.

En revanche, si le fichier existe toujours et que la configuration serveur n'a pas été corrigée, même le niveau 5 ne peut rien faire contre un fichier accessible par URL.

La meilleure garantie reste de supprimer/désactiver phpinfo() et d'ajouter une authentification côté serveur.

Q4 :

Dans le code source, le mécanisme de sécurité mis en œuvre vérifie si l'utilisateur est connecté avant d'afficher la page phpinfo().

Le script contrôle généralement la présence d'une session active ou d'un rôle autorisé.

Si l'utilisateur n'est pas authentifié, le code redirige vers la page de connexion ou affiche un message d'accès refusé.

Ce système empêche donc les visiteurs non identifiés d'accéder à des informations sensibles.

Cependant, tous les utilisateurs ne sont pas bloqués : seuls les administrateurs ou comptes autorisés peuvent y accéder.

Ainsi, la sécurité repose sur la gestion correcte des sessions et des droits d'accès.

Ce mécanisme limite les fuites d'informations, à condition que les contrôles soient bien appliqués.

Q5 :

Pour empêcher totalement l'accès à la page phpinfo() par le Web, il faut désactiver cette fonction dans la configuration PHP.

Dans le fichier php.ini situé dans /etc/php/7.0/apache2/, on peut ajouter ou modifier la directive suivante :

disable_functions = phpinfo

Cette ligne bloque l'exécution de la fonction phpinfo() pour tous les scripts PHP du serveur.

Ainsi, même si un utilisateur accède à la page phpinfo.php, la fonction ne sera plus exécutée.

Le serveur affichera alors une erreur ou une page vide.

Cette méthode est efficace, car elle agit au niveau de la configuration globale du serveur.

Elle protège même si le fichier phpinfo.php reste présent dans le répertoire Web.

Après la modification, il faut redémarrer Apache avec la commande sudo systemctl restart apache2.

Cela garantit qu'aucun utilisateur, y compris l'administrateur, ne puisse exécuter phpinfo() via le Web.

Travail à faire 3 :

Q1 :

Dans un premier temps, les trois machines ont été démarrées : le serveur Mutillidae, la machine attaquante avec BurpSuite et la machine victime.

La communication entre les différentes machines a été vérifiée afin de s'assurer qu'elles peuvent échanger correctement.

Les tests de connexion montrent que le réseau fonctionne normalement.

Une fois l'environnement prêt, l'application Mutillidae a été ouverte sur le serveur.

Le niveau de sécurité a ensuite été modifié.

Celui-ci a été réglé sur 0 afin de permettre la réalisation des attaques.

L'environnement de travail est maintenant prêt pour la suite du TP.

Travail à faire 4 :

Q1 :

Le niveau de sécurité 1 ne permet pas d'empêcher l'attaque. Malgré l'activation de ce premier niveau de sécurité, la faille est toujours exploitable et l'attaque fonctionne encore. Cela montre que ce niveau de protection est insuffisant pour sécuriser correctement la page.

Q2 :

Non, le niveau de sécurité 1 n'empêche pas une attaque MITM de type SSL-strip.

Ce niveau applique seulement des protections basiques au niveau de l'application web, pas de sécurité réseau forte.

Une attaque MITM exploite la couche réseau et la mauvaise gestion des redirections HTTP/HTTPS, ce qui échappe au simple réglage d'une appli.

Pour se protéger efficacement il faut des mesures réseau et navigateur : forcer HTTPS côté serveur, activer HSTS, et marquer les cookies Secure.

Il est aussi important de vérifier les certificats côté client et d'utiliser des réseaux de confiance (ou VPN).

La faille se situe principalement dans la configuration du transport (SSL/HTTP) : redirections mal faites (HTTP non forcé vers HTTPS) et absence de HSTS.

Aussi côté serveur : certificats mal installés/acceptation de connexions non sécurisées et cookies non marqués Secure/HttpOnly.

Côté réseau, un réseau non fiable (pas d'isolation) permet le positionnement d'un attaquant (MITM).

Q3 :

Le fichier index.php est le point d'entrée de l'application et initialise la session et les fichiers de configuration.

Il inclut les fonctions utilitaires et vérifie si un utilisateur est authentifié.

Selon le niveau de sécurité, il applique des contrôles simples (vérification de session, masquage d'informations).

Il gère les redirections internes vers les pages protégées ou la page de connexion.

Il effectue un échappement basique des sorties pour réduire certaines injections côté appli.

Il ne gère pas la sécurité réseau (SSL/TLS) : cette couche dépend de la configuration du serveur Apache.

Q4 :

Le niveau de sécurité 5 renforce fortement les protections de l'application et peut limiter certaines failles. Il applique des contrôles stricts sur les sessions, l'accès aux pages sensibles et l'échappement des données. Cependant une attaque MITM/SSL-strip exploite la couche réseau et la mauvaise configuration SSL, qui ne dépend pas du niveau d'appli. Ainsi même au niveau 5, si les redirections HTTPS ne sont pas correctement forcées ou si HSTS n'est pas activé, l'attaque reste possible. Pour être vraiment protégé il faut durcir la configuration SSL du serveur, forcer HTTPS et utiliser des cookies sécurisés. Le niveau 5 réduit les risques applicatifs, mais ne remplace pas une bonne configuration réseau et transport.

Q5 :

Non le niveau 5 ne suffit pas à empêcher complètement les conséquences de l'attaque. L'attaquant pourra toujours intercepter le trafic si le serveur n'a pas une bonne configuration. Les identifiants ou données sensibles peuvent être encore volés si HTTPS n'est pas forcé ou HSTS n'est pas activé. Le niveau 5 limite surtout les failles côté application, mais ne protège pas contre l'interception réseau.

Pour éviter les conséquences, il faut durcir la couche transport : forcer HTTPS, activer HSTS, utiliser des cookies Secure et vérifier les certificats.

Q6 :

Au niveau de sécurité 5, le fichier index.php applique des contrôles plus stricts sur l'accès aux pages et la gestion des sessions.

Il vérifie si l'utilisateur est bien authentifié avant d'autoriser l'accès à certaines parties du site.

En cas d'accès non autorisé ou d'attaque détectée, l'utilisateur est redirigé vers la page de connexion ou une page d'erreur.

Le code protège aussi mieux les données en filtrant et en validant les entrées.

Cependant, la sécurité réseau (comme HTTPS) dépend toujours de la configuration du serveur.

Cette vérification dans le code limite les fuites d'informations sensibles.

La bonne pratique à adopter est de forcer l'utilisation du HTTPS pour toutes les pages de l'application.

Il faut aussi activer le protocole HSTS et utiliser des cookies sécurisés.

Ces mesures garantissent la confidentialité des données échangées entre l'utilisateur et le serveur.